PRACTICAL NO – 1

A) WRITE A MONGODB QUERY TO CREATE, DISPLAY AND DROP A DATABASE.

- **CREATE DATABASE**

```
test> use mydb2
switched to db mydb2
```

- **DISPLAY DATABASE**

```
mydb2> show dbs
admin   40.00 KiB
config  72.00 KiB
local   96.00 KiB
mydb1   72.00 KiB
```

- **DROP DATABASE**

```
mydb2> db.dropDatabase()
{ ok: 1, dropped: 'mydb2' }
mydb2> db
mydb2
```

B) WRITE A MONGODB QUERY TO CREATE, DISPLAY AND DROP A COLLECTION.

- **CREATE COLLECTION**

```
mydb2> db.createCollection("flights")
{ ok: 1 }
```

- **DISPLAY COLLECTION**

```
mydb2> show collections
flights
```

- **DROP COLLECTION**

```
mydb2> db.flights.drop()
true
mydb2> show collections
```

C) **WRITE A MONGODB QUERY TO INSERT A DOCUMENT INTO THE COLLECTION (insertOne , insertMany).**

- **INSERT ONE**

CODE:

```
mydb2> db.flights.insertOne(
...       {"departureAirport":"MUM",
...        "arrivalAirport":"DLH",
...        "aircraft":"AirbusA380",
...        "distance":12000,
...        "intercontinental":"false"}
...      )
```

OUTPUT:

```
{
  acknowledged: true,
  insertedId: ObjectId('6927b87b8d97cde56c63b112')
}
```

- **INSERT MANY**

CODE:

```
mydb2> db.flights.insertMany([
...       {"departureAirport":"MUM",
...        "arrivalAirport":"BLR",
...        "aircraft":"AirbusA123",
...        "distance":14000,
...        "intercontinental":"false"},
...       {"departureAirport":"MUM",
...        "arrivalAirport":"BLR",
...        "aircraft":"AirbusA567",
...        "distance":14000,
...        "intercontinental":"false"},
...       {"departureAirport":"MUM",
...        "arrivalAirport":"BLR",
...        "aircraft":"AirbusA789",
...        "distance":14000,
...        "intercontinental":"false"},
...       {"departureAirport":"MUM",
...        "arrivalAirport":"DBI",
...        "aircraft":"AirbusA222",
...        "distance":34000,
...        "intercontinental":"true"},
...       {"departureAirport":"MUM",
...        "arrivalAirport":"GRY",
...        "aircraft":"AirbusA567",
...        "distance":84000,
...        "intercontinental":"true"}]
...    )
```

OUTPUT:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6927bb668d97cde56c63b113'),
    '1': ObjectId('6927bb668d97cde56c63b114'),
    '2': ObjectId('6927bb668d97cde56c63b115'),
    '3': ObjectId('6927bb668d97cde56c63b116'),
    '4': ObjectId('6927bb668d97cde56c63b117')
  }
}
```

D) WRITE A MONGODB QUERY TO RETRIEVE THE DOCUMENTS (find, findOne).

- **FIND**

CODE:

```
mydb2> db.flights.findOne()
```

OUTPUT:

```
{
  _id: ObjectId('6927b87b8d97cde56c63b112'),
  departureAirport: 'MUM',
  arrivalAirport: 'DLH',
  aircraft: 'AirbusA380',
  distance: 12000,
  intercontinental: 'false'
}
```

- **FIND ONE**

CODE:

```
mydb2> db.flights.find()
```

OUTPUT:

```
[
  {
    _id: ObjectId('6927b87b8d97cde56c63b112'),
    departureAirport: 'MUM',
    arrivalAirport: 'DLH',
    aircraft: 'AirbusA380',
    distance: 12000,
    intercontinental: 'false'
  },
  {
    _id: ObjectId('6927bb668d97cde56c63b113'),
    departureAirport: 'MUM',
    arrivalAirport: 'BLR',
    aircraft: 'AirbusA123',
    distance: 14000,
    intercontinental: 'false'
  },
  {
    _id: ObjectId('6927bb668d97cde56c63b114'),
    departureAirport: 'MUM',
    arrivalAirport: 'BLR',
    aircraft: 'AirbusA567',
    distance: 14000,
    intercontinental: 'false'
  },
  {
    _id: ObjectId('6927bb668d97cde56c63b115'),
    departureAirport: 'MUM',
    arrivalAirport: 'BLR',
    aircraft: 'AirbusA789',
    distance: 14000,
    intercontinental: 'false'
```

```
  },
  {
    _id: ObjectId('6927bb668d97cde56c63b116'),
    departureAirport: 'MUM',
    arrivalAirport: 'DBI',
    aircraft: 'AirbusA222',
    distance: 34000,
    intercontinental: 'true'
  },
  {
    _id: ObjectId('6927bb668d97cde56c63b117'),
    departureAirport: 'MUM',
    arrivalAirport: 'GRY',
    aircraft: 'AirbusA567',
    distance: 84000,
    intercontinental: 'true'
  }
]
```

E) WRITE A MONGODB QUERY TO UPDATE THE DOCUMENTS (updateOne, updateMany).

- **UPDATE ONE**

CODE:

```
mydb2> db.flights.updateOne(
...       {"departureAirport":"MUM"},
...       {$set:{"aircraft":"AirbusA777"}}
...      )
```

OUTPUT:

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
mydb2> db.flights.find()
[
  {
    _id: ObjectId('6927b87b8d97cde56c63b112'),
    departureAirport: 'MUM',
    arrivalAirport: 'DLH',
    aircraft: 'AirbusA777',
    distance: 12000,
    intercontinental: 'false'
  },
```

- **UPDATE MANY**

CODE:

```
mydb2> db.flights.updateMany(
...       {"departureAirport":"MUM"},
...       {$set:{"aircraft":"AirbusA777"}}
...      )
```

OUTPUT:

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 6,
  modifiedCount: 5,
  upsertedCount: 0
}
```

```
mydb2> db.flights.find()
[
  {
    _id: ObjectId('6927b87b8d97cde56c63b112'),
    departureAirport: 'MUM',
    arrivalAirport: 'DLH',
    aircraft: 'AirbusA777',
    distance: 12000,
    intercontinental: 'false'
  },
  {
    _id: ObjectId('6927bb668d97cde56c63b113'),
    departureAirport: 'MUM',
    arrivalAirport: 'BLR',
    aircraft: 'AirbusA777',
    distance: 14000,
    intercontinental: 'false'
  },
  {
    _id: ObjectId('6927bb668d97cde56c63b114'),
    departureAirport: 'MUM',
    arrivalAirport: 'BLR',
    aircraft: 'AirbusA777',
    distance: 14000,
    intercontinental: 'false'
  },
  {
    _id: ObjectId('6927bb668d97cde56c63b115'),
    departureAirport: 'MUM',
    arrivalAirport: 'BLR',
    aircraft: 'AirbusA777',
    distance: 14000,
    intercontinental: 'false'
  },
```

```
  {
    _id: ObjectId('6927bb668d97cde56c63b116'),
    departureAirport: 'MUM',
    arrivalAirport: 'DBI',
    aircraft: 'AirbusA777',
    distance: 34000,
    intercontinental: 'true'
  },
  {
    _id: ObjectId('6927bb668d97cde56c63b117'),
    departureAirport: 'MUM',
    arrivalAirport: 'GRY',
    aircraft: 'AirbusA777',
    distance: 84000,
    intercontinental: 'true'
  }
]
```

F) WRITE A MONGODB QUERY TO DELETE THE DOCUMENTS (deleteOne, deleteMany).

- **DELETE ONE**

CODE:

```
mydb2> db.flights.deleteOne(
...       {"intercontinental":"false"}
...      )
```

OUTPUT:

```
{ acknowledged: true, deletedCount: 1 }
```

- **DELETE MANY**

CODE:

```
mydb2> db.flights.deleteMany(
...       {"intercontinental":"true"}
...      )
```

OUTPUT:

```
{ acknowledged: true, deletedCount: 2 }
```

PRACTICAL NO – 2

1. **Create a collection users and insert documents into it (as per the files attached).**

CODE:

```
test> use mydb2
switched to db mydb2
mydb2> db.createCollection("users")
{ ok: 1 }
mydb2> db.users.insertMany([
...   {
...     "name":"Max Miller","age":29,"gender":"M","country":"IND"
...   },
...   {
...     "name":"Manu Lorenz","age":30,"gender":"F","country":"US"
...   },
...   {
...     "name":"Chris Hayton","age":35,"gender":"M","country":"US"
...   },
...   {
...     "name":"Sandeep Kumar","age":28,"gender":"M","country":"IND"
...   },
...   {
...     "name":"Maria Jones","age":30,"gender":"F","country":"US"
...   },
...   {
...     "name":"Alexandra Maier","age":27,"gender":"F","country":"UK"
...   },
...   {
...     "name":"Dr.Phil Evans","age":47,"gender":"M","country":"US"
...   },
...   {
...     "name":"Sandra John","age":33,"gender":"F","country":"UK"
...   },
...   {
...     "name":"Elisa George","age":29,"gender":"F","country":"US"
...   },
```

```
...   {
...     "name":"Frank Cube","age":41,"gender":"M","country":"UK"
...   },
...   {
...     "name":"Karandeep Alun","age":48,"gender":"M","country":"IND"
...   },
...   {
...     "name":"Michaela Drayer","age":39,"gender":"F","country":"UK"
...   },
...   {
...     "name":"Bernd Hoftstadt","age": 22,"gender":"M","country":"AUS"
...   },
...   {
...     "name":"Scott Tolib","age":44,"gender":"M","country":"AUS"
...   },
...   {
...     "name":"Freddy Melver","age": 41,"gender":"M","country":"UK"
...   },
...   {
...     "name":"Alex John","age": 35,"gender":"M","country":"IND"
...   },
...   {
...     "name":"Melanie Palace","age": 27,"gender":"F","country":"AUS"
...   },
...   {
...     "name":"Armin Glutch","age":35,"gender":"M","country":"US"
...   },
...   {
...     "name":"Klaus Arber","age":53,"gender":"M","country":"UK"
...   },
...   {
...     "name":"Albert Twostone","age":68,"gender":"M","country":"US"
...   },
...   {
```

```
...    {
...      "name":"Gordon Black","age":38,"gender":"F","country":"US"
...    },
... {
...     "name":"Sanjay Khan","age":38,"gender":"M","country":"IND"
... },
... {
...      "name":"Priya Joshi","age":38, "gender":"F","country":"IND"
... },
... {
...      "name":"Sanjana Ray","age":31,"gender":"F","country":"IND"
... },
... {
...      "name":"Anjali Mehta","age": 40,"gender":"F","country":"IND"
... }
... ])
```

OUTPUT:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69310c82ec2ae1d52263b112'),
    '1': ObjectId('69310c82ec2ae1d52263b113'),
    '2': ObjectId('69310c82ec2ae1d52263b114'),
    '3': ObjectId('69310c82ec2ae1d52263b115'),
    '4': ObjectId('69310c82ec2ae1d52263b116'),
    '5': ObjectId('69310c82ec2ae1d52263b117'),
    '6': ObjectId('69310c82ec2ae1d52263b118'),
    '7': ObjectId('69310c82ec2ae1d52263b119'),
    '8': ObjectId('69310c82ec2ae1d52263b11a'),
    '9': ObjectId('69310c82ec2ae1d52263b11b'),
    '10': ObjectId('69310c82ec2ae1d52263b11c'),
    '11': ObjectId('69310c82ec2ae1d52263b11d'),
    '12': ObjectId('69310c82ec2ae1d52263b11e'),
    '13': ObjectId('69310c82ec2ae1d52263b11f'),
    '14': ObjectId('69310c82ec2ae1d52263b120'),
    '15': ObjectId('69310c82ec2ae1d52263b121'),
    '16': ObjectId('69310c82ec2ae1d52263b122'),
    '17': ObjectId('69310c82ec2ae1d52263b123'),
    '18': ObjectId('69310c82ec2ae1d52263b124'),
    '19': ObjectId('69310c82ec2ae1d52263b125'),
    '20': ObjectId('69310c82ec2ae1d52263b126'),
    '21': ObjectId('69310c82ec2ae1d52263b127'),
    '22': ObjectId('69310c82ec2ae1d52263b128'),
    '23': ObjectId('69310c82ec2ae1d52263b129'),
    '24': ObjectId('69310c82ec2ae1d52263b12a')
  }
}
```

2. **List all the documents in users collection.**

CODE:

```
mydb2> db.users.find()
```

OUTPUT:

```
  {
    _id: ObjectId('69310c82ec2ae1d52263b112'),
    name: 'Max Miller',
    age: 29,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b113'),
    name: 'Manu Lorenz',
    age: 30,
    gender: 'F',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b114'),
    name: 'Chris Hayton',
    age: 35,
    gender: 'M',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b115'),
    name: 'Sandeep Kumar',
    age: 28,
    gender: 'M',
    country: 'IND'
  },
```

```
  {
    _id: ObjectId('69310c82ec2ae1d52263b116'),
    name: 'Maria Jones',
    age: 30,
    gender: 'F',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b117'),
    name: 'Alexandra Maier',
    age: 27,
    gender: 'F',
    country: 'UK'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b118'),
    name: 'Dr.Phil Evans',
    age: 47,
    gender: 'M',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b119'),
    name: 'Sandra John',
    age: 33,
    gender: 'F',
    country: 'UK'
  },
```

```
  {
    _id: ObjectId('69310c82ec2ae1d52263b11a'),
    name: 'Elisa George',
    age: 29,
    gender: 'F',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b11b'),
    name: 'Frank Cube',
    age: 41,
    gender: 'M',
    country: 'UK'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b11c'),
    name: 'Karandeep Alun',
    age: 48,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b11d'),
    name: 'Michaela Drayer',
    age: 39,
    gender: 'F',
    country: 'UK'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b11e'),
    name: 'Bernd Hoftstadt',
    age: 22,
    gender: 'M',
    country: 'AUS'
  },
```

```
  {
    _id: ObjectId('69310c82ec2ae1d52263b122'),
    name: 'Melanie Palace',
    age: 27,
    gender: 'F',
    country: 'AUS'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b123'),
    name: 'Armin Glutch',
    age: 35,
    gender: 'M',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b124'),
    name: 'Klaus Arber',
    age: 53,
    gender: 'M',
    country: 'UK'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b125'),
    name: 'Albert Twostone',
    age: 68,
    gender: 'M',
    country: 'US'
  }
]
Type "it" for more
```

3. **Projectors (Include/Exclude specific fields).**

a. **Select only the name and age fileds from the users collection.**

CODE:

```
mydb2> db.users.find({},{name:1,age:1,_id:0})
```

OUTPUT:

```
[
  { name: 'Max Miller', age: 29 },
  { name: 'Manu Lorenz', age: 30 },
  { name: 'Chris Hayton', age: 35 },
  { name: 'Sandeep Kumar', age: 28 },
  { name: 'Maria Jones', age: 30 },
  { name: 'Alexandra Maier', age: 27 },
  { name: 'Dr.Phil Evans', age: 47 },
  { name: 'Sandra John', age: 33 },
  { name: 'Elisa George', age: 29 },
  { name: 'Frank Cube', age: 41 },
  { name: 'Karandeep Alun', age: 48 },
  { name: 'Michaela Drayer', age: 39 },
  { name: 'Bernd Hoftstadt', age: 22 },
  { name: 'Scott Tolib', age: 44 },
  { name: 'Freddy Melver', age: 41 },
  { name: 'Alex John', age: 35 },
  { name: 'Melanie Palace', age: 27 },
  { name: 'Armin Glutch', age: 35 },
  { name: 'Klaus Arber', age: 53 },
  { name: 'Albert Twostone', age: 68 }
]
Type "it" for more
```

b. Display all the fields in the users collection except, _id, age and gender.

CODE:

```
mydb2> db.users.find({},{_id:0,age:0,gender:0})
```

OUTPUT:

```
[
  { name: 'Max Miller', country: 'IND' },
  { name: 'Manu Lorenz', country: 'US' },
  { name: 'Chris Hayton', country: 'US' },
  { name: 'Sandeep Kumar', country: 'IND' },
  { name: 'Maria Jones', country: 'US' },
  { name: 'Alexandra Maier', country: 'UK' },
  { name: 'Dr.Phil Evans', country: 'US' },
  { name: 'Sandra John', country: 'UK' },
  { name: 'Elisa George', country: 'US' },
  { name: 'Frank Cube', country: 'UK' },
  { name: 'Karandeep Alun', country: 'IND' },
  { name: 'Michaela Drayer', country: 'UK' },
  { name: 'Bernd Hoftstadt', country: 'AUS' },
  { name: 'Scott Tolib', country: 'AUS' },
  { name: 'Freddy Melver', country: 'UK' },
  { name: 'Alex John', country: 'IND' },
  { name: 'Melanie Palace', country: 'AUS' },
  { name: 'Armin Glutch', country: 'US' },
  { name: 'Klaus Arber', country: 'UK' },
  { name: 'Albert Twostone', country: 'US' }
]
Type "it" for more
```

4. **Selectors, AND, OR operations.**

a. **Fetch all the Indian users from the list.**

CODE:

```
mydb2> db.users.find({country:"IND"})
```

OUTPUT:

```
[
  {
    _id: ObjectId('69310c82ec2ae1d52263b112'),
    name: 'Max Miller',
    age: 29,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b115'),
    name: 'Sandeep Kumar',
    age: 28,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b11c'),
    name: 'Karandeep Alun',
    age: 48,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b121'),
    name: 'Alex John',
    age: 35,
    gender: 'M',
    country: 'IND'
  },
```

```
  {
    _id: ObjectId('69310c82ec2ae1d52263b128'),
    name: 'Priya Joshi',
    age: 38,
    gender: 'F',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b129'),
    name: 'Sanjana Ray',
    age: 31,
    gender: 'F',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b12a'),
    name: 'Anjali Mehta',
    age: 40,
    gender: 'F',
    country: 'IND'
  }
]
```

b. Fetch all the female users from US.

CODE:

```
mydb2> db.users.find({$and:[{gender:"F"},{country:"US"}]})
```

OUTPUT:

```
[
  {
    _id: ObjectId('69310c82ec2ae1d52263b113'),
    name: 'Manu Lorenz',
    age: 30,
    gender: 'F',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b116'),
    name: 'Maria Jones',
    age: 30,
    gender: 'F',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b11a'),
    name: 'Elisa George',
    age: 29,
    gender: 'F',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b126'),
    name: 'Gordon Black',
    age: 38,
    gender: 'F',
    country: 'US'
  }
]
```

c. **Fetch all the Indian and US users from the list.**

CODE:

```
mydb2> db.users.find({$or:[{country:"IND"},{country:"US"}]})
```

OUTPUT:

```
[
  {
    _id: ObjectId('69310c82ec2ae1d52263b112'),
    name: 'Max Miller',
    age: 29,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b113'),
    name: 'Manu Lorenz',
    age: 30,
    gender: 'F',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b114'),
    name: 'Chris Hayton',
    age: 35,
    gender: 'M',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b115'),
    name: 'Sandeep Kumar',
    age: 28,
    gender: 'M',
    country: 'IND'
  },
```

```
  {
    _id: ObjectId('69310c82ec2ae1d52263b127'),
    name: 'Sanjay Khan',
    age: 38,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b128'),
    name: 'Priya Joshi',
    age: 38,
    gender: 'F',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b129'),
    name: 'Sanjana Ray',
    age: 31,
    gender: 'F',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b12a'),
    name: 'Anjali Mehta',
    age: 40,
    gender: 'F',
    country: 'IND'
  }
]
```

d. Fetch all the US users with age=30 from the list.

CODE:

```
mydb2> db.users.find({$and:[{country:"US"},{age:30}]})
```

OUTPUT:

```
[
  {
    _id: ObjectId('69310c82ec2ae1d52263b113'),
    name: 'Manu Lorenz',
    age: 30,
    gender: 'F',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b116'),
    name: 'Maria Jones',
    age: 30,
    gender: 'F',
    country: 'US'
  }
]
```

5. **Combining Selectors and Projectors.**

a. **Display only name, age & country of Indian Users.**

CODE:

```
mydb2> db.users.find({country:"IND"},{name:1,age:1,country:1,_id:0})
```

OUTPUT:

```
[
  { name: 'Max Miller', age: 29, country: 'IND' },
  { name: 'Sandeep Kumar', age: 28, country: 'IND' },
  { name: 'Karandeep Alun', age: 48, country: 'IND' },
  { name: 'Alex John', age: 35, country: 'IND' },
  { name: 'Sanjay Khan', age: 38, country: 'IND' },
  { name: 'Priya Joshi', age: 38, country: 'IND' },
  { name: 'Sanjana Ray', age: 31, country: 'IND' },
  { name: 'Anjali Mehta', age: 40, country: 'IND' }
]
```

b. **Fetch the name and country of all male users.**

CODE:

```
mydb2> db.users.find({gender:"M"},{name:1,country:1,_id:0})
```

OUTPUT:

```
[
  { name: 'Max Miller', country: 'IND' },
  { name: 'Chris Hayton', country: 'US' },
  { name: 'Sandeep Kumar', country: 'IND' },
  { name: 'Dr.Phil Evans', country: 'US' },
  { name: 'Frank Cube', country: 'UK' },
  { name: 'Karandeep Alun', country: 'IND' },
  { name: 'Bernd Hoftstadt', country: 'AUS' },
  { name: 'Scott Tolib', country: 'AUS' },
  { name: 'Freddy Melver', country: 'UK' },
  { name: 'Alex John', country: 'IND' },
  { name: 'Armin Glutch', country: 'US' },
  { name: 'Klaus Arber', country: 'UK' },
  { name: 'Albert Twostone', country: 'US' },
  { name: 'Sanjay Khan', country: 'IND' }
]
```

6. **Sort(), limit(), skip(), count().**

a. **Fetch the name and age of all male users in ascending order of age.**

CODE:

```
mydb2> db.users.find({gender:"M"},{name:1,age:1,_id:0}).sort({age:1})
```

OUTPUT:

```
[
  { name: 'Bernd Hoftstadt', age: 22 },
  { name: 'Sandeep Kumar', age: 28 },
  { name: 'Max Miller', age: 29 },
  { name: 'Chris Hayton', age: 35 },
  { name: 'Alex John', age: 35 },
  { name: 'Armin Glutch', age: 35 },
  { name: 'Sanjay Khan', age: 38 },
  { name: 'Frank Cube', age: 41 },
  { name: 'Freddy Melver', age: 41 },
  { name: 'Scott Tolib', age: 44 },
  { name: 'Dr.Phil Evans', age: 47 },
  { name: 'Karandeep Alun', age: 48 },
  { name: 'Klaus Arber', age: 53 },
  { name: 'Albert Twostone', age: 68 }
]
```

b. **Fetch the name and age of all male users in descending order of name and ascending order of age.**

CODE:

```
mydb2> db.users.find({gender:"M"},{name:1,age:1,_id:0}).sort({name:-1,age:1})
```

OUTPUT:

```
[
  { name: 'Scott Tolib', age: 44 },
  { name: 'Sanjay Khan', age: 38 },
  { name: 'Sandeep Kumar', age: 28 },
  { name: 'Max Miller', age: 29 },
  { name: 'Klaus Arber', age: 53 },
  { name: 'Karandeep Alun', age: 48 },
  { name: 'Freddy Melver', age: 41 },
  { name: 'Frank Cube', age: 41 },
  { name: 'Dr.Phil Evans', age: 47 },
  { name: 'Chris Hayton', age: 35 },
  { name: 'Bernd Hoftstadt', age: 22 },
  { name: 'Armin Glutch', age: 35 },
  { name: 'Alex John', age: 35 },
  { name: 'Albert Twostone', age: 68 }
]
```

c. **Fetch the first 5 male users**

CODE:

```
mydb2> db.users.find({gender:"M"}).limit(5)
```

OUTPUT:

```
[
  {
    _id: ObjectId('69310c82ec2ae1d52263b112'),
    name: 'Max Miller',
    age: 29,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b114'),
    name: 'Chris Hayton',
    age: 35,
    gender: 'M',
    country: 'US'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b115'),
    name: 'Sandeep Kumar',
    age: 28,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b118'),
    name: 'Dr.Phil Evans',
    age: 47,
    gender: 'M',
    country: 'US'
  },
```

```
  {
    _id: ObjectId('69310c82ec2ae1d52263b11b'),
    name: 'Frank Cube',
    age: 41,
    gender: 'M',
    country: 'UK'
  }
]
```

d. **Fetch the male users from India. Skip the first two records and list the next three records.**

CODE:

```
mydb2> db.users.find({$and:[{gender:"M"},{country:"IND"}]}).limit(3).skip(2)
```

OUTPUT:

```
[
  {
    _id: ObjectId('69310c82ec2ae1d52263b11c'),
    name: 'Karandeep Alun',
    age: 48,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b121'),
    name: 'Alex John',
    age: 35,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b127'),
    name: 'Sanjay Khan',
    age: 38,
    gender: 'M',
    country: 'IND'
  }
]
```

e. **Fetch the male users from India. Skip the first two records**

CODE:

```
mydb2> db.users.find({$and:[{gender:"M"},{country:"IND"}]}).skip(2)
```

OUTPUT:

```
[
  {
    _id: ObjectId('69310c82ec2ae1d52263b11c'),
    name: 'Karandeep Alun',
    age: 48,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b121'),
    name: 'Alex John',
    age: 35,
    gender: 'M',
    country: 'IND'
  },
  {
    _id: ObjectId('69310c82ec2ae1d52263b127'),
    name: 'Sanjay Khan',
    age: 38,
    gender: 'M',
    country: 'IND'
  }
]
```

f. Count the total number of users

CODE:

```
mydb2> db.users.find().count()
```

OUTPUT:

```
25
```

g. Count the total number of female users in US

CODE:

```
mydb2> db.users.find({$and:[{gender:"F"},{country:"US"}]}).count()
```

OUTPUT:

```
4
```

PRACTICAL NO – 3

1. Using Conditional and Logical Operators AND, OR, NOT.

Create the Employees Collection and Insert Documents

CODE:

```
test> use mydb2
switched to db mydb2
mydb2> db.createCollection("Employees")
{ ok: 1 }
mydb2> db.Employees.insertMany([
...   { name: "Alice", age: 30, department: "HR", salary: 50000, joinDate: new Date("2015-01-15") },
...   { name: "Bob", age: 24, department: "Engineering", salary: 70000, joinDate: new Date("2019-03-10") },
...   { name: "Charlie", age: 29, department: "Engineering", salary: 75000, joinDate: new Date("2017-06-23") },
...   { name: "David", age: 35, department: "Marketing", salary: 60000, joinDate: new Date("2014-11-01") },
...   { name: "Eve", age: 28, department: "Finance", salary: 80000, joinDate: new Date("2018-08-19") }
... ])
```

OUTPUT:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('693a2b2c87eb8101bb63b112'),
    '1': ObjectId('693a2b2c87eb8101bb63b113'),
    '2': ObjectId('693a2b2c87eb8101bb63b114'),
    '3': ObjectId('693a2b2c87eb8101bb63b115'),
    '4': ObjectId('693a2b2c87eb8101bb63b116')
  }
}
```

a. Find employees in the “Engineering” department.

CODE:

```
mydb2> db.Employees.find({department: {$eq: "Engineering"}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a2b2c87eb8101bb63b113'),
    name: 'Bob',
    age: 24,
    department: 'Engineering',
    salary: 70000,
    joinDate: ISODate('2019-03-10T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b114'),
    name: 'Charlie',
    age: 29,
    department: 'Engineering',
    salary: 75000,
    joinDate: ISODate('2017-06-23T00:00:00.000Z')
  }
]
```

b. Find employees who are not in the "HR" department.

CODE:

```
mydb2> db.Employees.find({department:{$ne: "HR"}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a2b2c87eb8101bb63b113'),
    name: 'Bob',
    age: 24,
    department: 'Engineering',
    salary: 70000,
    joinDate: ISODate('2019-03-10T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b114'),
    name: 'Charlie',
    age: 29,
    department: 'Engineering',
    salary: 75000,
    joinDate: ISODate('2017-06-23T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b115'),
    name: 'David',
    age: 35,
    department: 'Marketing',
    salary: 60000,
    joinDate: ISODate('2014-11-01T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b116'),
    name: 'Eve',
    age: 28,
    department: 'Finance',
    salary: 80000,
    joinDate: ISODate('2018-08-19T00:00:00.000Z')
  }
```

c. Find employees who are older than 30.

CODE:

```
mydb2> db.Employees.find({age:{$gt: 30}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a2b2c87eb8101bb63b115'),
    name: 'David',
    age: 35,
    department: 'Marketing',
    salary: 60000,
    joinDate: ISODate('2014-11-01T00:00:00.000Z')
  }
]
```

d. Find employees with a salary less than 70000.

CODE:

```
mydb2> db.Employees.find({salary:{$lt: 70000}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a2b2c87eb8101bb63b112'),
    name: 'Alice',
    age: 30,
    department: 'HR',
    salary: 50000,
    joinDate: ISODate('2015-01-15T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b115'),
    name: 'David',
    age: 35,
    department: 'Marketing',
    salary: 60000,
    joinDate: ISODate('2014-11-01T00:00:00.000Z')
  }
]
```

e. Find employees who joined on or after January 1, 2018

CODE:

```
mydb2> db.Employees.find({joinDate:{$gte: new Date("2018-01-01")}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a2b2c87eb8101bb63b113'),
    name: 'Bob',
    age: 24,
    department: 'Engineering',
    salary: 70000,
    joinDate: ISODate('2019-03-10T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b116'),
    name: 'Eve',
    age: 28,
    department: 'Finance',
    salary: 80000,
    joinDate: ISODate('2018-08-19T00:00:00.000Z')
  }
]
```

f. Find employees who are 28 years old or younger.

CODE:

```
mydb2> db.Employees.find({age:{$lte: 28}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a2b2c87eb8101bb63b113'),
    name: 'Bob',
    age: 24,
    department: 'Engineering',
    salary: 70000,
    joinDate: ISODate('2019-03-10T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b116'),
    name: 'Eve',
    age: 28,
    department: 'Finance',
    salary: 80000,
    joinDate: ISODate('2018-08-19T00:00:00.000Z')
  }
]
```

g. $and (Logical AND)

Find employees who are in the "Engineering" department and have a salary greater than 70000.

CODE:

```
mydb2> db.Employees.find({$and:[{department: "Engineering"}, {salary: {$gt: 70000}}]})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a2b2c87eb8101bb63b114'),
    name: 'Charlie',
    age: 29,
    department: 'Engineering',
    salary: 75000,
    joinDate: ISODate('2017-06-23T00:00:00.000Z')
  }
]
```

h. $or (Logical OR)

Find employees who are in the "HR" department and have a salary less than 60000.

CODE:

```
mydb2> db.Employees.find({$or:[{department: "HR"}, {salary: {$lt: 60000}}]})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a2b2c87eb8101bb63b112'),
    name: 'Alice',
    age: 30,
    department: 'HR',
    salary: 50000,
    joinDate: ISODate('2015-01-15T00:00:00.000Z')
  }
]
```

i. **$not (Logical NOT)**

Find employees who are not in the "Finance" department.

CODE:

```
mydb2> db.Employees.find({department: {$not: {$eq: "Finance"}}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a2b2c87eb8101bb63b112'),
    name: 'Alice',
    age: 30,
    department: 'HR',
    salary: 50000,
    joinDate: ISODate('2015-01-15T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b113'),
    name: 'Bob',
    age: 24,
    department: 'Engineering',
    salary: 70000,
    joinDate: ISODate('2019-03-10T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b114'),
    name: 'Charlie',
    age: 29,
    department: 'Engineering',
    salary: 75000,
    joinDate: ISODate('2017-06-23T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b115'),
    name: 'David',
    age: 35,
    department: 'Marketing',
    salary: 60000,
    joinDate: ISODate('2014-11-01T00:00:00.000Z')
  }
]
```

j. $nor (Logical NOR)

Find employees who are neither in the "HR" department nor have a salary greater than 75000.

CODE:

```
mydb2> db.Employees.find({$nor:[{department: "HR"},{salary: {$gt: "75000"}}]})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a2b2c87eb8101bb63b113'),
    name: 'Bob',
    age: 24,
    department: 'Engineering',
    salary: 70000,
    joinDate: ISODate('2019-03-10T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b114'),
    name: 'Charlie',
    age: 29,
    department: 'Engineering',
    salary: 75000,
    joinDate: ISODate('2017-06-23T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b115'),
    name: 'David',
    age: 35,
    department: 'Marketing',
    salary: 60000,
    joinDate: ISODate('2014-11-01T00:00:00.000Z')
  },
  {
    _id: ObjectId('693a2b2c87eb8101bb63b116'),
    name: 'Eve',
    age: 28,
    department: 'Finance',
    salary: 80000,
    joinDate: ISODate('2018-08-19T00:00:00.000Z')
  }
]
```

2. Query Selectors

2b. Bitwise Selectors

$bitsAllClear: Matches documents where all specified bit positions have a value of 0

$bitsAllSet: Matches documents where all specified bit positions have a value of 1

$bitsAnyClear: Matches documents where any of the specified bit positions have a value of 0

$bitsAnySet: Matches documents where any of the specified bit positions have a value of 1

CODE:

```
geodb> use mydb2
switched to db mydb2
mydb2> db.createCollection("Devices")
{ ok: 1 }
mydb2> db.Devices.insertMany([
...   { name: "Device A", status: 5 }, // Binary: 0101
...   { name: "Device B", status: 3 }, // Binary: 0011
...   { name: "Device C", status: 12 }, // Binary: 1100
...   { name: "Device D", status: 10 }, // Binary: 1010
...   { name: "Device E", status: 7 }  // Binary: 0111
... ])
```

OUTPUT:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('693a3d3287eb8101bb63b11c'),
    '1': ObjectId('693a3d3287eb8101bb63b11d'),
    '2': ObjectId('693a3d3287eb8101bb63b11e'),
    '3': ObjectId('693a3d3287eb8101bb63b11f'),
    '4': ObjectId('693a3d3287eb8101bb63b120')
  }
}
```

a. **Find devices where the binary status has both the 1st and 3rd bits set**

CODE:

```
mydb2> db.Devices.find({status: {$bitsAllSet: [0,2]}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a3d3287eb8101bb63b11c'),
    name: 'Device A',
    status: 5
  },
  {
    _id: ObjectId('693a3d3287eb8101bb63b120'),
    name: 'Device E',
    status: 7
  }
]
```

b. **Find devices where the binary status has at least the 2nd bit set**

CODE:

```
mydb2> db.Devices.find({status: {$bitsAnySet: [1]}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a3d3287eb8101bb63b11d'),
    name: 'Device B',
    status: 3
  },
  {
    _id: ObjectId('693a3d3287eb8101bb63b11f'),
    name: 'Device D',
    status: 10
  },
  {
    _id: ObjectId('693a3d3287eb8101bb63b120'),
    name: 'Device E',
    status: 7
  }
]
```

c. **Find devices where the binary status has both the 2nd and 4th bits clear**

CODE:

```
mydb2> db.Devices.find({status: {$bitsAllClear: [1,3]}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a3d3287eb8101bb63b11c'),
    name: 'Device A',
    status: 5
  }
]
```

d. **Find devices where the binary status has at least the 1st bit clear**

CODE:

```
mydb2> db.Devices.find({status: {$bitsAllClear: [0]}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('693a3d3287eb8101bb63b11e')
    name: 'Device C',
    status: 12
  },
  {
    _id: ObjectId('693a3d3287eb8101bb63b11f')
    name: 'Device D',
    status: 10
  }
]
```

PRACTICAL NO – 4

1. Projection Operators ($, $elematch and $slice)

a. Create the Products Collection and Insert Documents

CODE:

```
test> use mydb2
switched to db mydb2
mydb2> db.createCollection("Products")
{ ok: 1 }
mydb2> db.Products.insertMany([
...   {
...     name: "Laptop",
...     brand: "BrandA",
...     features: [
...       { name: "Processor", value: "Intel i7" },
...       { name: "RAM", value: "16GB" },
...       { name: "Storage", value: "512GB SSD" }
...     ],
...     reviews: [
...       { user: "Alice", rating: 5, comment: "Excellent!" },
...       { user: "Bob", rating: 4, comment: "Very good" },
...       { user: "Charlie", rating: 3, comment: "Average" }
...     ]
...   },
...   {
...     name: "Smartphone",
...     brand: "BrandB",
...     features: [
...       { name: "Processor", value: "Snapdragon 888" },
...       { name: "RAM", value: "8GB" },
...       { name: "Storage", value: "256GB" }
...     ],
...     reviews: [
...       { user: "Dave", rating: 4, comment: "Good phone" },
...       { user: "Eve", rating: 2, comment: "Not satisfied" }
...     ]
...   },
... {
```

```
...     name: "iPad",
...     brand: "BrandC",
...     features: [
...       { name: "Processor", value: "A14 Bionic" },
...       { name: "RAM", value: "16GB" },
...       { name: "Storage", value: "1 TB" }
...     ],
...     reviews: [
...       { user: "Arya", rating: 5, comment: "Excellent!" },
...       { user: "Babita", rating: 4, comment: "Very good" },
...       { user: "Chetan", rating: 3, comment: "Average" }
...     ]
...   }
... ])
```

OUTPUT:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('694367b32936390aeb63b112'),
    '1': ObjectId('694367b32936390aeb63b113'),
    '2': ObjectId('694367b32936390aeb63b114')
  }
}
```

b. Find the product named "Laptop" and project the review from the user "Alice".

CODE:

```
mydb2> db.Products.find({name:"Laptop","reviews.user":"Alice"},{"reviews.$":1})
```

OUTPUT:

```
[
  {
    _id: ObjectId('694367b32936390aeb63b112'),
    reviews: [ { user: 'Alice', rating: 5, comment: 'Excellent!' } ]
  }
]
```

c. Find the product named "Laptop" and project the review where the rating is greater than 4.

CODE:

```
mydb2> db.Products.find({name:"Laptop"},{reviews: {$elemMatch: {rating: {$gt:4}}}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('694367b32936390aeb63b112'),
    reviews: [ { user: 'Alice', rating: 5, comment: 'Excellent!' } ]
  }
]
```

d. Find the product named "Smartphone" and project the first review.

CODE:

```
mydb2> db.Products.find({name:"Smartphone"},{reviews: {$slice:1}})
```

OUTPUT:

```
[
  {
    _id: ObjectId('694367b32936390aeb63b113'),
    name: 'Smartphone',
    brand: 'BrandB',
    features: [
      { name: 'Processor', value: 'Snapdragon 888' },
      { name: 'RAM', value: '8GB' },
      { name: 'Storage', value: '256GB' }
    ],
    reviews: [ { user: 'Dave', rating: 4, comment: 'Good phone' } ]
  }
]
```

e. **Find the product named "Laptop" and project the name, the first two features, and the review with the highest rating.**

CODE:

```
mydb2> db.Products.find({name: "Laptop"},
... {
...  name: 1,_id:0,
... features: {$slice:2},
... reviews: {$elemMatch: {rating:5}}
... }
... )
```

OUTPUT:

```
[
  {
    name: 'Laptop',
    features: [
      { name: 'Processor', value: 'Intel i7' },
      { name: 'RAM', value: '16GB' }
    ],
    reviews: [ { user: 'Alice', rating: 5, comment: 'Excellent!' } ]
  }
]
```

2. **Aggregation operations such as $sum, $avg, $min, $max, $push, and $addToSet**

a. **Create the Products Collection and Insert Documents**

CODE:

```
mydb2> db.createCollection("Sales")
{ ok: 1 }
mydb2> db.Sales.insertMany([
... {date: new Date("2024-01-01"), product:"Laptop", price:12000, quantity: 1, customer: "Amar"},
... {date: new Date("2024-01-02"), product:"Laptop", price:18000, quantity: 2, customer: "Babu"},
... {date: new Date("2024-01-03"), product:"Mouse", price:2500, quantity: 5, customer: "Chandra"},
... {date: new Date("2024-01-04"), product:"Keyboard", price:4500, quantity: 3, customer: "Amar"},
... {date: new Date("2024-01-05"), product:"Monitor", price:30000, quantity: 1, customer: "Babu"},
... {date: new Date("2024-01-06"), product:"Laptop", price:12000, quantity: 1, customer: "Deva"},
... ])
```

OUTPUT:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69436ede2936390aeb63b115'),
    '1': ObjectId('69436ede2936390aeb63b116'),
    '2': ObjectId('69436ede2936390aeb63b117'),
    '3': ObjectId('69436ede2936390aeb63b118'),
    '4': ObjectId('69436ede2936390aeb63b119'),
    '5': ObjectId('69436ede2936390aeb63b11a')
  }
}
```

b. **Calculate the total price of each product.**

CODE:

```
mydb2> db.Sales.aggregate([
...   {
...    $group: {
...    _id:"$product",
...    totalPrice:{$sum:"$price"}
...    }
...   }
... ])
```

OUTPUT:

```
[
  { _id: 'Keyboard', totalPrice: 4500 },
  { _id: 'Monitor', totalPrice: 30000 },
  { _id: 'Laptop', totalPrice: 42000 },
  { _id: 'Mouse', totalPrice: 2500 }
]
```

c. Calculate the average price of each product.

CODE:

```
mydb2> db.Sales.aggregate([
...   {
...    $group: {
...    _id:"$product",
...    averagePrice:{$avg:"$price"}
...    }
...   }
... ])
```

OUTPUT:

```
[
  { _id: 'Monitor', averagePrice: 30000 },
  { _id: 'Keyboard', averagePrice: 4500 },
  { _id: 'Laptop', averagePrice: 14000 },
  { _id: 'Mouse', averagePrice: 2500 }
]
```

d. Find the minimum price of each product.

CODE:

```
mydb2> db.Sales.aggregate([
...   {
...    $group: {
...    _id:"$product",
...    minPrice:{$min:"$price"}
...    }
...   }
... ])
```

OUTPUT:

```
[
  { _id: 'Monitor', minPrice: 30000 },
  { _id: 'Mouse', minPrice: 2500 },
  { _id: 'Laptop', minPrice: 12000 },
  { _id: 'Keyboard', minPrice: 4500 }
]
```

e. Find the maximum price of each product.

CODE:

```
mydb2> db.Sales.aggregate([
...   {
...    $group: {
...    _id:"$product",
...    maxPrice:{$max:"$price"}
...    }
...   }
... ])
```

OUTPUT:

```
[
  { _id: 'Keyboard', maxPrice: 4500 },
  { _id: 'Monitor', maxPrice: 30000 },
  { _id: 'Laptop', maxPrice: 18000 },
  { _id: 'Mouse', maxPrice: 2500 }
]
```

PRACTICAL NO – 5

1. **Explain() with indexes.**

a. **Explain() with all three modes for male users in users collection.**

CODE:

```
mydb1> db.users.find({"gender":"M"}).explain("allPlansExecution")
```

OUTPUT:

```
{
  queryPlanner: {
    plannerVersion: 1,
    namespace: 'mydb1.users',
    indexFilterSet: false,
    parsedQuery: { gender: { '$eq': 'M' } },
    winningPlan: {
      stage: 'COLLSCAN',
      filter: { gender: { '$eq': 'M' } },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 14,
    executionTimeMillis: 13,
    totalKeysExamined: 0,
    totalDocsExamined: 25,
    executionStages: {
      stage: 'COLLSCAN',
      filter: { gender: { '$eq': 'M' } },
      nReturned: 14,
      executionTimeMillisEstimate: 11,
      works: 27,
      advanced: 14,
      needTime: 12,
      needYield: 0,
      saveState: 1,
      restoreState: 1,
      isEOF: 1,
      direction: 'forward',
      docsExamined: 25
    },
    allPlansExecution: []
  },
  serverInfo: {
    host: 'DESKTOP-QOCTQUQ',
    port: 27017,
    version: '4.4.1',
    gitVersion: 'ad91a93a5a31e175f5cbf8c69561e788bbc55ce1'
  },
  ok: 1
}
```

CODE:

```
mydb1> db.users.find({"gender":"M"}).explain("executionStats")
```

OUTPUT:

```
{
  queryPlanner: {
    plannerVersion: 1,
    namespace: 'mydb1.users',
    indexFilterSet: false,
    parsedQuery: { gender: { '$eq': 'M' } },
    winningPlan: {
      stage: 'COLLSCAN',
      filter: { gender: { '$eq': 'M' } },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 14,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 25,
    executionStages: {
      stage: 'COLLSCAN',
      filter: { gender: { '$eq': 'M' } },
      nReturned: 14,
      executionTimeMillisEstimate: 0,
      works: 27,
      advanced: 14,
      needTime: 12,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      direction: 'forward',
      docsExamined: 25
    }
  },
  serverInfo: {
    host: 'DESKTOP-QOCTQUQ',
    port: 27017,
    version: '4.4.1',
    gitVersion: 'ad91a93a5a31e175f5cbf8c69561e788bbc55ce1'
  },
  ok: 1
}
```

CODE:

```
mydb1> db.users.find({"gender":"M"}).explain("queryPlanner")
```

OUTPUT:

```
{
  queryPlanner: {
    plannerVersion: 1,
    namespace: 'mydb1.users',
    indexFilterSet: false,
    parsedQuery: { gender: { '$eq': 'M' } },
    queryHash: '1F945B0D',
    planCacheKey: '1F945B0D',
    winningPlan: {
      stage: 'COLLSCAN',
      filter: { gender: { '$eq': 'M' } },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  serverInfo: {
    host: 'DESKTOP-QOCTQUQ',
    port: 27017,
    version: '4.4.1',
    gitVersion: 'ad91a93a5a31e175f5cbf8c69561e788bbc55ce1'
  },
  ok: 1
}
```

b. Single Key Index

CODE:

```
mydb1> db.users.ensureIndex({"name":1})
```

OUTPUT:

```
[ 'name_1' ]
```

c. Compound Key Index

CODE:

```
mydb1> db.users.ensureIndex({"name":1,"age":1})
```

OUTPUT:

```
[ 'name_1_age_1' ]
```

d. View Index

CODE:

```
mydb1> db.users.getIndexes()
```

OUTPUT:

```
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1' },
  { v: 2, key: { name: 1, age: 1 }, name: 'name_1_age_1' }
]
```

e. Delete Index

CODE:

```
mydb1> db.users.dropIndex({"name":1})
```

OUTPUT:

```
{ nIndexesWas: 3, ok: 1 }
```

PRACTICAL NO – 6

❖ **JQUERY SELECTORS**

1. ELEMENT NAME SELECTOR

CODE:

```
<html>

<head>
    <title> The Selector Example</title>
    <script type="text/javascript" src="jquery-3.7.1.min.js"></script>

    <script type="text/javascript" language="javascript">
        $(document).ready(function () {
            $("div").css("background-color", "yellow");
        });
    </script>
</head>

<body>

    <div class="big" id="div1">
        <p>This is first division of the DOM</p>
    </div>

    <div class="medium" id="div2">
        <p>This is second division of the DOM</p>
    </div>

    <div class="small" id="div3">
        <p>This is third division of the DOM</p>
    </div>
</body>

</html>
```

OUTPUT:

This is first division of the DOM

This is second division of the DOM

This is third division of the DOM

2. ELEMENT ID SELECTOR

CODE:

```
<html>

<head>
    <title> The Selector Example</title>
    <script type="text/javascript" src="jquery-3.7.1.min.js"></script>

    <script type="text/javascript" language="javascript">
        $(document).ready(function () {
            $("#div2").css("background-color", "yellow");
        });
    </script>
</head>

<body>

    <div class="big" id="div1">
        <p>This is first division of the DOM</p>
    </div>

    <div class="medium" id="div2">
        <p>This is second division of the DOM</p>
    </div>

    <div class="small" id="div3">
        <p>This is third division of the DOM</p>
    </div>
</body>

</html>
```

OUTPUT:

This is first division of the DOM

This is second division of the DOM

This is third division of the DOM

3. ELEMENT CLASS SELECTOR

CODE:

```
<html>

<head>
    <title> The Selector Example</title>
    <script type="text/javascript" src="jquery-3.7.1.min.js"></script>

    <script type="text/javascript" language="javascript">
        $(document).ready(function () {
            $(".big").css("background-color", "yellow");
        });
    </script>
</head>

<body>

    <div class="big" id="div1">
        <p>This is first division of the DOM</p>
    </div>

    <div class="medium" id="div2">
        <p>This is second division of the DOM</p>
    </div>

    <div class="small" id="div3">
        <p>This is third division of the DOM</p>
    </div>
</body>

</html>
```

OUTPUT:

This is first division of the DOM

This is second division of the DOM

This is third division of the DOM

4. UNIVERSAL SELECTOR

CODE:

```
<html>

<head>
    <title> The Selector Example</title>
    <script type="text/javascript" src="jquery-3.7.1.min.js"></script>

    <script type="text/javascript" language="javascript">
        $(document).ready(function () {
            $("*").css("background-color", "yellow");
        });
    </script>
</head>

<body>

    <div class="big" id="div1">
        <p>This is first division of the DOM</p>
    </div>

    <div class="medium" id="div2">
        <p>This is second division of the DOM</p>
    </div>

    <div class="small" id="div3">
        <p>This is third division of the DOM</p>
    </div>
</body>

</html>
```

OUTPUT:

This is first division of the DOM

This is second division of the DOM

This is third division of the DOM

5. MULTIPLE ELEMENT SELECTOR

CODE:

```
<html>

<head>
    <title> The Selector Example</title>
    <script type="text/javascript" src="jquery-3.7.1.min.js"></script>

    <script type="text/javascript" language="javascript">
        $(document).ready(function () {
            $(".big, #div3").css("background-color", "yellow");
        });
    </script>
</head>

<body>

    <div class="big" id="div1">
        <p>This is first division of the DOM</p>
    </div>

    <div class="medium" id="div2">
        <p>This is second division of the DOM</p>
    </div>

    <div class="small" id="div3">
        <p>This is third division of the DOM</p>
    </div>
</body>

</html>
```

OUTPUT:

This is first division of the DOM

This is second division of the DOM

This is third division of the DOM

PRACTICAL NO – 7

6. CLICK EVENT

CODE: (event.html)

```html
<html>
    <head>
        <title>Click Event</title>
        <script type="text/javascript" src="jquery-3.7.1.min.js"></script>
        <script type="text/javascript" src="app.js"></script>
        <link rel="stylesheet" type="text/css" href="style.css"/>
    </head>

    <body>
        <p>Click on any square below</p>
        <div class = "div" style="background-color: blue;">ONE</div>
        <div class = "div" style="background-color: green;">TWO</div>
        <div class = "div" style="background-color: red;">THREE</div>
    </body>
</html>
```

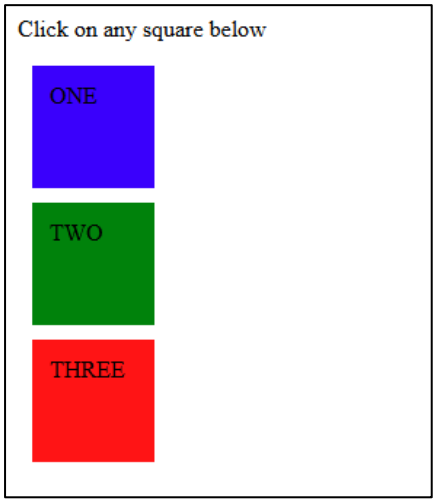
CODE: (style.css)

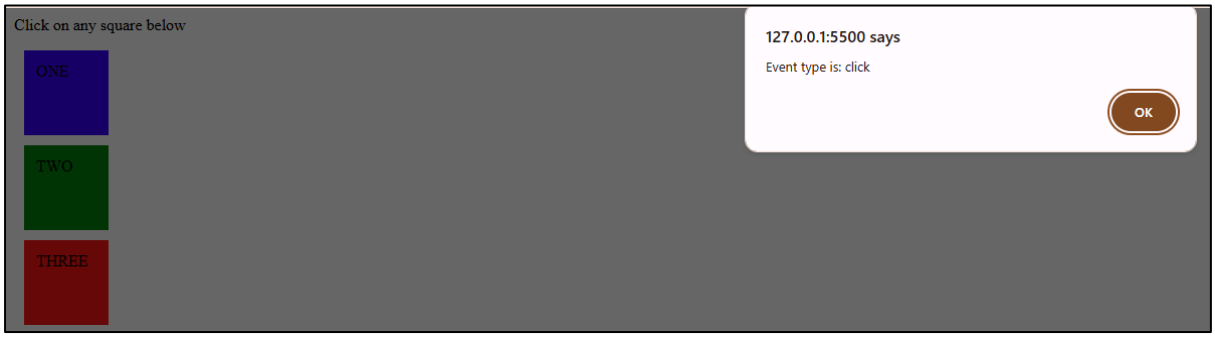
```css
.div{
    margin:10px;
    padding:12px;
    border:2px;
    width:60px;
    height:60px;
}
```

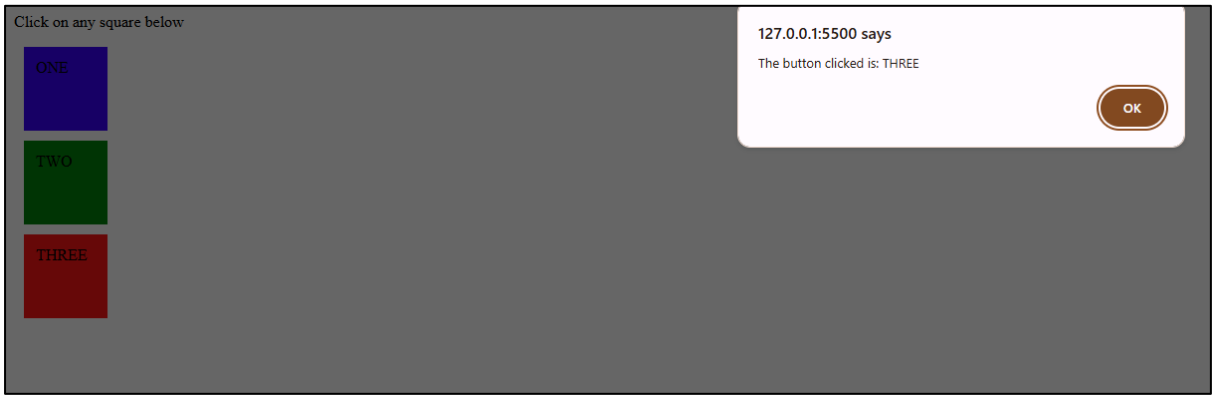
CODE: (app.js)

```javascript
$(document).ready(function(){
$("div").click(function()
{
    alert('Event type is: '+event.type);
    alert('The button clicked is: '+event.target.innerHTML);
});
});
```

OUTPUT:


Click on any square below
ONE
TWO
THREE


Click on any square below
ONE
TWO
THREE
127.0.0.1:5500 says
Event type is: click
OK


Click on any square below
ONE
TWO
THREE
127.0.0.1:5500 says
The button clicked is: THREE
OK

7. DOUBLE CLICK EVENT

CODE: (event.html)

```html
<html>
    <head>
        <title>Double Click Event</title>
        <script type="text/javascript" src="jquery-3.7.1.min.js"></script>
        <script type="text/javascript" src="app.js"></script>
        <link rel="stylesheet" type="text/css" href="style.css"/>
    </head>

    <body>
        <p>Click on any square below to hide it</p>
        <div class = "div" style="background-color: blue;">ONE</div>
        <div class = "div" style="background-color: green;">TWO</div>
        <div class = "div" style="background-color: red;">THREE</div>
    </body>
</html>
```

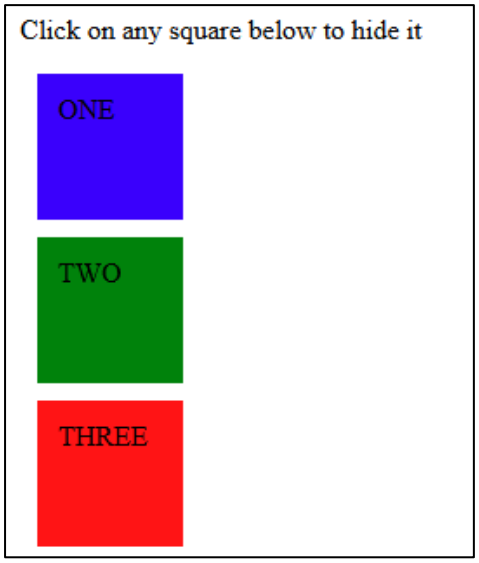
CODE: (style.css)

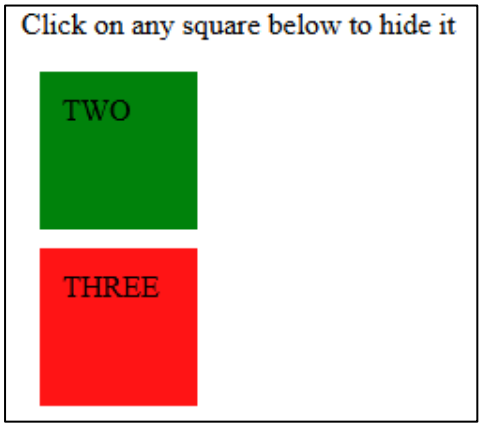
```css
.div{
    margin:10px;
    padding:12px;
    border:2px;
    width:60px;
    height:60px;
}
```

CODE: (app.js)

```javascript
$(document).ready(function(){
$("div").dblclick(function(){
    $(this).hide();
});
});
```

OUTPUT:


Click on any square below to hide it
ONE
TWO
THREE


Click on any square below to hide it
TWO
THREE

8. FADING EFFECT

CODE: (effect.html)

```html
<html>
    <head>
        <title>Fading Effect</title>
        <script type="text/javascript" src="jquery-3.7.1.min.js"></script>
        <script type="text/javascript" src="app.js"></script>
        <link rel="stylesheet" type="text/css" href="style.css"/>
    </head>

    <body>
        <p>Click on any of the buttons</p>
        <button id = "out"> FADE OUT</button>
        <button id = "in"> FADE IN</button>
        <div class = "pic"> <img src="sies.jpeg" alt="jQuery"/></div>
        <div class = "log"></div>
    </body>
</html>
```

CODE: (style.css)

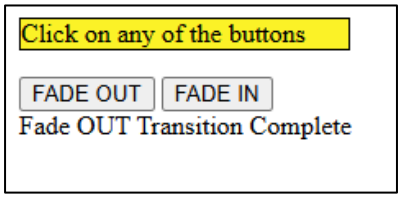
```css
p{
    background-color: rgb(255, 238, 0);
    width:200px;
    border:1px solid;
}

img{
    height:150px;
    width:150px;
}
```

CODE: (app.js)

```javascript
$(document).ready(function(){
    $("#in").click(function(){
        $(".pic").fadeIn('slow',function(){
            $(".log").text("Fade In Tarnsition Complete");
        });
    });
    $("#out").click(function(){
        $(".pic").fadeOut('slow',function(){
            $(".log").text("Fade OUT Tarnsition Complete");
        });
    });
});
```

OUTPUT:

Click on any of the buttons
FADE OUT
FADE IN
SIES


Click on any of the buttons
FADE OUT
FADE IN
Fade OUT Transition Complete

Click on any of the buttons
FADE OUT
FADE IN
SIES
Fade In Transition Complete

9. SLIDING EFFECT

CODE: (effect.html)

```html
<html>
    <head>
        <title>Sliding Effect</title>
        <script type="text/javascript" src="jquery-3.7.1.min.js"></script>
        <script type="text/javascript" src="app.js"></script>
        <link rel="stylesheet" type="text/css" href="style.css"/>
    </head>

    <body>
        <p>Click on any of the buttons</p>
        <button id = "up"> SLIDE UP</button>
        <button id = "down"> SLIDE DOWN</button>
        <div class = "pic"> <img src="sies.jpeg" alt="jQuery"/></div>
        <div class = "log"></div>
    </body>
</html>
```

CODE: (style.css)

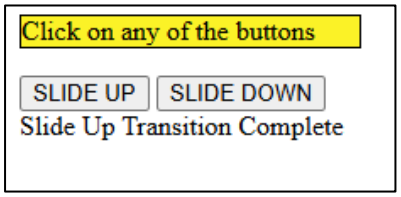
```css
p{
    background-color: rgb(255, 238, 0);
    width:200px;
    border:1px solid;
}

img{
    height:150px;
    width:150px;
}
```

CODE: (app.js)

```javascript
$(document).ready(function(){
    $("#up").click(function(){
        $(".pic").slideUp('slow',function(){
            $(".log").text("Slide Up Transition Complete");
        });
    });
    $("#down").click(function(){
        $(".pic").slideDown('slow',function(){
            $(".log").text("Slide Out Transition Complete");
        });
    });
});
```

OUTPUT:

Click on any of the buttons
SLIDE UP
SLIDE DOWN
SIES


Click on any of the buttons
SLIDE UP
SLIDE DOWN
Slide Up Transition Complete

Click on any of the buttons
SLIDE UP
SLIDE DOWN
SIES
Slide Out Transition Complete

10. ANIMATION EFFECT

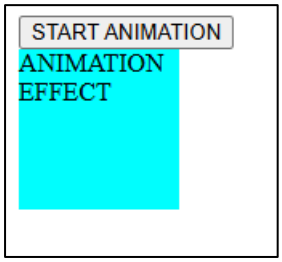
<u>CODE:</u> (effect.html)

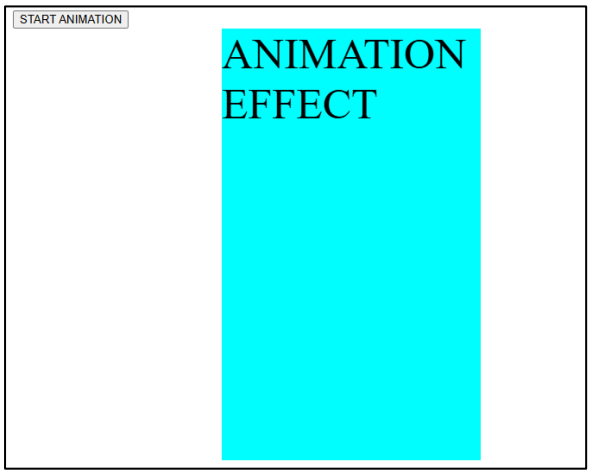
```html
<html>
    <head>
        <title>Animation Effect</title>
        <script type="text/javascript" src="jquery-3.7.1.min.js"></script>
        <script type="text/javascript" src="app.js"></script>
        <link rel="stylesheet" type="text/css" href="style.css"/>
    </head>

    <body>
        <button> START ANIMATION</button>
        <div style="background-color: aqua;
        height: 100px;
        width:100px;
        position:absolute;">ANIMATION EFFECT
        </div>
    </body>
</html>
```

<u>CODE:</u> (app.js)

```javascript
$(document).ready(function(){
$("button").click(function()
{
    $("div").animate({left:'250px'},"slow");
    $("div").animate({height:'500px'},"slow");
    $("div").animate({width:'300px'},"slow");
    $("div").animate({fontSize:'50px'},"slow");
});
});
```

OUTPUT:


START ANIMATION
ANIMATION
EFFECT

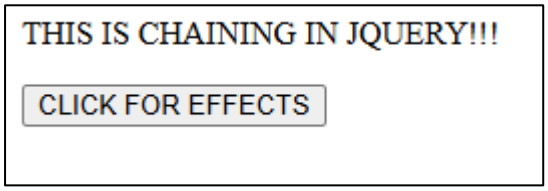
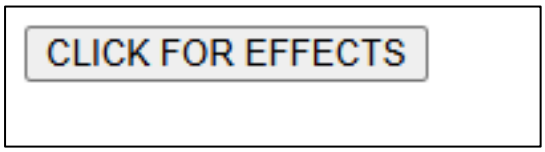
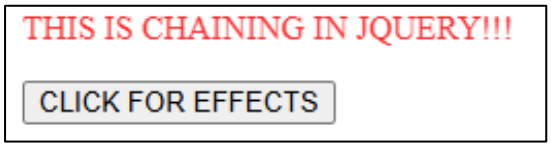

START ANIMATION
ANIMATION
EFFECT

11. CHAINING EFFECT

CODE: (effect.html)

```html
<html>
    <head>
        <title>Chaining Effect</title>
        <script type="text/javascript" src="jquery-3.7.1.min.js"></script>
        <script type="text/javascript" src="app.js"></script>
        <link rel="stylesheet" type="text/css" href="style.css"/>
    </head>

    <body>
        <p id="txt">THIS IS CHAINING IN JQUERY!!!</p>
        <button>CLICK FOR EFFECTS</button>
    </body>
</html>
```

CODE: (app.js)

```javascript
$(document).ready(function(){
$("button").click(function()
{
    $("#txt").css("color","red").slideUp(1000).slideDown(3000).fadeOut(3000).f
adeIn(3000);
});
});
```

OUTPUT: